ChatGPT

Next.js Portfolio Project Roadmap

Tech Stack & Setup

Start by creating a Next.js project (preferably v13+). Next.js provides built-in static generation and SEO-friendly file-based routing [1]. Install Tailwind CSS and configure it (enable JIT mode, dark mode, etc.), and import the **Inter** font as the site's base typography [2]. Optionally use TypeScript for type safety and set up ESLint/Prettier for consistent code style.

- Initialize with `npx create-next-app` (or the App Router template).
- Install Tailwind and configure `tailwind.config.js` (enable `darkMode: 'class'`).
- Add the Inter font (via a `<link>` to Google Fonts or by importing in `globals.css`) [2].
- (Optional) Add TypeScript support and linting tools (as done in some portfolios for maintainability [3]).

Architecture & Folder Structure

Use a modular folder structure: put all UI components in one place and static content in another. Each major section (Hero, About, Projects, etc.) should be a self-contained component [4]. For example, have a top-level `components/` folder containing `Navbar.tsx`, `Footer.tsx`, `HeroSection.tsx`, `ProjectCard.tsx`, etc., and a `data/` or `content/` folder containing JSON or Markdown for projects and posts [5]. Place shared assets (images, CV PDF) in `public/`.

- **Folder setup:** `src/components/` for React components; `src/data/` or `content/` for static project/blog data; `public/` for static assets (images, CV PDF).
- Define a global `Layout` component (`app/layout.tsx` or `pages/_app.js`) to wrap pages. Import Tailwind's base styles and the Inter font here.
- Use path aliases (e.g. `@components/`) for cleaner imports.

This modular structure makes updates easier: changing one component or data file won't affect unrelated parts of the code [6].

Layout & Core Components

Build the site's shell components first. Create a **Navbar** (or Header) component with links (Home, About, Projects, Blog, Contact) and a "Download CV" button that links to your resume PDF. Include a **DarkModeToggle** in the nav. Create a **Footer** component with your social links (LinkedIn, GitHub, Email) using icon buttons. All pages/sections will be wrapped by this layout.

- **Navbar.tsx:** Sticky at top, with site navigation links (anchors to sections) and a CTA button for the CV (e.g. an `<a href="/resume.pdf" download>`). Include a dark mode toggle switch.
- **Footer.tsx:** Simple footer with icons linking to LinkedIn, GitHub, and mailto. Use semantically labeled elements (e.g. `<footer>`) and aria-labels for icons.
- **Layout.tsx:** Global layout that includes `<Navbar>` and `<Footer>`, and applies global styles (Tailwind classes, dark mode class on `<html>`).

Hero Section

The Hero section is the first thing visitors see. Create a **HeroSection** component that displays your name, a brief tagline, and primary CTA buttons ("Download CV" and "Contact Me"). Use a responsive layout (e.g. flex or grid) to position text on one side and an illustrative image or icon on the other. For example, one design put the introduction text on the left and a profile image on the right, with a centered CTA button [7].

- **HeroSection.tsx:** Large heading for name, subheading/tagline, and two buttons (CV and Contact). Add subtle hover animations on buttons.
- **CV Button:** An `<a>` styled as a button, linking to a static `resume.pdf` in `public/`.
- **Responsiveness:** On mobile, stack the content vertically; on desktop, use a two-column layout.

This anchors attention and guides the user immediately to key actions [7].

About Section

Create an **AboutSection** component to summarize your background. Include a short bio paragraph and education credentials. This could be a simple card or timeline. Keep it concise and emphasize key points. For example, list degrees and institutions, along with a sentence or two about your interests or background.

- **AboutSection.tsx:** Title ("About Me"), brief introduction text.
- **Education List:** A list or timeline of education entries (e.g. degree, school, year). Data can come from a JSON/constants file for easy updates.

Aim for clarity and readability (e.g. plenty of whitespace, semantic headings).

Projects Section

The **ProjectsSection** showcases your work. Store project data (title, description, tech tags, image URL, links) in a separate file (e.g. `content/projects.js`) so it's easy to update [8]. Create a **ProjectCard** component that takes these props and renders a card with image, title, description, and links (e.g. GitHub, demo). Add tag badges (Web, Mobile, AI/ML, etc.) on each card.

- **ProjectCard.tsx:** Displays a project's image, title, short description, and a set of tag badges. Include buttons or links for "Demo" and "Source".
- **ProjectsSection.tsx:** Imports the array of projects and renders a grid of `<ProjectCard>`s.
- **Filtering (optional):** Add a tag filter UI (e.g. a row of toggle buttons above the grid) to filter projects by category.

This section should be visually clean and allow users to quickly scan your work. Update the project list by editing the data file (no code change required) [8].

Blog Section

Add a **BlogSection** (even with placeholder content). If you have articles, use static Markdown or JSON to list them. Create a **BlogCard** component for each post with title, excerpt, and link (to a full post page or external). For now, static links are fine.

- **BlogCard.tsx:** Shows post title, snippet, and a "Read More" link.
- **BlogSection.tsx:** Displays a list (or grid) of recent posts using `<BlogCard>`.

Having a blog section is recommended ("includes blogs to showcase understanding" [9]), even if content is minimal initially.

Contact Section

The **ContactSection** lets visitors reach out. You can either provide a simple contact form or just list your contact info. One design puts social summary on the left and a form on the right [10]. For simplicity, you can include a short text ("Get in touch") and a basic form (name, email, message) using controlled React inputs (with simple client-side validation). Alternatively, just display your email as a mailto link and social icons.

- **ContactSection.tsx:** Title ("Contact"), brief invite text.
- **Contact Form (optional):** Input fields for Name, Email, Message, and a Submit button. (Without backend, the form might just trigger an email link.)
- **Or simple info:** A paragraph with your email and links (or social icons as in the footer).

Ensure labels and inputs are accessible (use `aria-label` and `type="email"`). A clean, uncluttered layout is key [10].

Footer & Social Links

Finally, ensure the **Footer** from the layout includes up-to-date social links. It can be very minimal (e.g. "©2025 Your Name" and icons). Keep it elegant and simple. In earlier portfolios, the footer was "simple, elegant, and complete" [11]. Make sure navigation (if any) is responsive and that the footer always stays at the bottom of the page.

- **Footer:** Personal logo or initials, current year, and links.
- Ensure links open in new tabs when appropriate (e.g. LinkedIn) and use `rel="noopener"`.

Styling & Theming

Use Tailwind utility classes for layout and spacing. Enable dark mode (e.g. Tailwind's `class` strategy) so users can toggle themes. A component library can speed up styling: for example, **DaisyUI** is a "free component library" for Tailwind that includes themeable components [12]. **Shadcn/ui** offers 200+ copy-paste Tailwind components (with TypeScript and Radix under the hood) [13]. **Headless UI** provides unstyled accessible primitives (dialogs, menus, etc.) if you need more customized controls [14].

- **Global styles:** In `globals.css`, set default background and text colors for light/dark modes. Import any global fonts.
- **Theme toggle:** Create a `<ThemeSwitch>` component that toggles the `dark` class on `<html>`; store preference in `localStorage`.

- **Responsive design:** Use Tailwind breakpoints (e.g. `sm:`, `md:`) to adjust layouts. One portfolio refactored grids into flex stacks and "optimized responsiveness with Tailwind utilities" [15].

Set the font family to Inter (`font-sans` with `@import` of Inter) for all text. Consistent spacing and color usage (e.g. a neutral light/dark scheme with one accent color) will keep the UI clean.

Animations & Interactivity

Add subtle animations and transitions for polish (hover states, button presses). Keep these lightweight to maintain performance: as one developer advises, use fewer animations since "the less JS, the better" [16]. For example, use CSS `transition` for buttons and simple fade/slide effects on sections. Optionally, use **Framer Motion** for fluid animations (it's used in some portfolios [17]).

- **Hover/Focus states:** Buttons and links should have clear hover effects (e.g. color or shadow changes).
- **Section reveals:** A slight fade-in when scrolling into view (using CSS or AOS library).
- **Avoid heavy effects:** Do not block page load with large animation libraries; keep payload small.

Minimal, purposeful animations keep the site "smooth and consistent" across devices [18].

Testing & Optimization

Finally, test each component individually (e.g. in Storybook or Jest/Cypress). Verify responsiveness on mobile/tablet: one author made sure "the mobile version wasn't just scaled-down... equally effective and intuitive" [18]. Check accessibility (semantic HTML, `alt` text, keyboard focus) since "details matter—especially in accessibility and responsiveness" [19]. Use browser devtools or Lighthouse to audit performance and SEO.

- **Responsive checks:** Resize the browser to ensure no overflow and good layout.
- **Accessibility:** Test with a screen reader or accessibility checker.
- **Performance:** Measure load times (Next.js static props help, images use `<Image>` for optimization).
- **Code quality:** Use ESLint/Prettier (as in [26]) and clean commit history.

Deploy when ready (Vercel is a natural choice). Each section/component should now be modular, easy to update, and work seamlessly together, showcasing a clean, minimal portfolio with light/dark support and polished UI [6] [15].

Sources: The above plan follows best practices from recent developer portfolio examples [4] [1] and Tailwind-based UI libraries [13] [12]. Each section/component recommendation is drawn from those sources to ensure a maintainable, aesthetic site.

---

[1] [3] [4] [6] [7] [10] [11] [15] [18] [19] A Deep Dive into My Developer Portfolio: Building personal website from the Ground Up | by Ramadhanu | Jul, 2025 | Medium
https://medium.com/@rdhanurzid/a-deep-dive-into-my-developer-portfolio-building-personal-website-from-the-ground-up-ad5cc5f7b341

[2] Inter font family
https://rsms.me/inter/

5 8 9 Open source Next.js Developer Portfolio Template [Modern + Responsive] - DEV Community
https://dev.to/abdulbasit313/open-source-nextjs-developer-portfolio-template-modern-responsive-42cj

12 Build and Deploy Your Modern Portfolio with Next.js, Tailwind CSS, DaisyUI, and Vercel ! — Tailwind CSS Components (version 5 update is here)
https://daisyui.com/resources/videos/build-and-deploy-your-modern-portfolio-with-nextjs-tailwind-css-daisyui-and-vercel-lwabaf3bky4/?lang=en

13 17 Portfolio Templates - Free shadcn/ui Components | shadcn.io - Shadcn.io
https://www.shadcn.io/template/category/portfolio

14 Headless UI - Unstyled, fully accessible UI components
https://headlessui.com/

16 Building your Next(.js) Portfolio Website | by Abhishek | Medium
https://medium.com/@abhisheksriram845/building-your-next-js-portfolio-website-f41f51b9c664